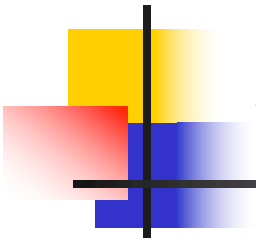


言語処理系 第1回
木村担当分の説明

木村啓二
keiji@waseda.jp

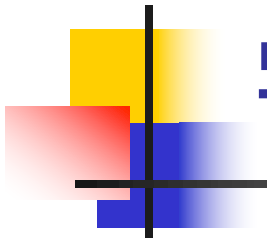
木村担当分

- フロントエンドとバックエンド
  - フロントエンド：ソースコードを読み取ってコンパイラの内部データ構造に落とし込む
  - バックエンド：コード生成
- **予習必須**
- **授業中、指していきます**
- 演習問題
  - 講義日中にMoodle上で回答します
- 実習・レポート
  - 簡単な電卓とコンパイラをお題としたレポート課題を出す予定
  - 全て提出の必要あり
  - 質問はMoodleのフォーラムに
- 実習環境**（本日この後配布）**
  - Raspberry Pi4にLinuxをインストールして使用
    - 必要なパッケージは実習説明のスライドで指定します



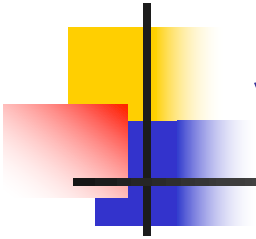
本パートの目標

- コンパイラのフロントエンドを理解する
  - 役割と限界
- プログラミング言語の形式的表現とそれを処理する方法の理解
  - 記号列の表現方法とそれを処理するメカニズムへの変換方法
- lex(flex), yacc(bison)の動作原理の理解
- そこそこの分量のコードを触ってみる
- 最適化処理の基本の理解
  - 何ができて何が苦手か



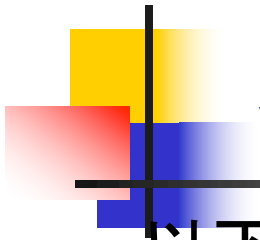
学習内容の応用範囲

- もちろん，通常のコンパイラ
  - gcc, llvm等の動作や構造の理解に有用
- 特定用途用のコンパイラもある
  - 最近は深層学習向けの研究・開発が活発
    - TVM, NVIDIA TensorRT等
  - これらの中でやっている処理を理解した上で活用できる
- コンパイラに直接関係なくとも，入力テキストファイルに従い何らかの処理を行うアプリケーションはいくらでもある



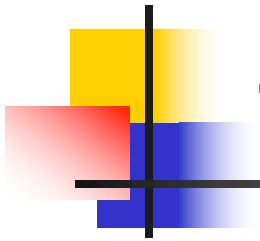
参考書

- 「コンパイラの構成と最適化」、中田育男、朝倉書店、9500円、ISBN-13: 978-4254121391
  - 第2版も出ています
- “Compilers: Principles, Techniques, & Tools (2nd Edition)”, A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, Addison Wesley
  - 邦訳もでていたような



木村担当分のトピック

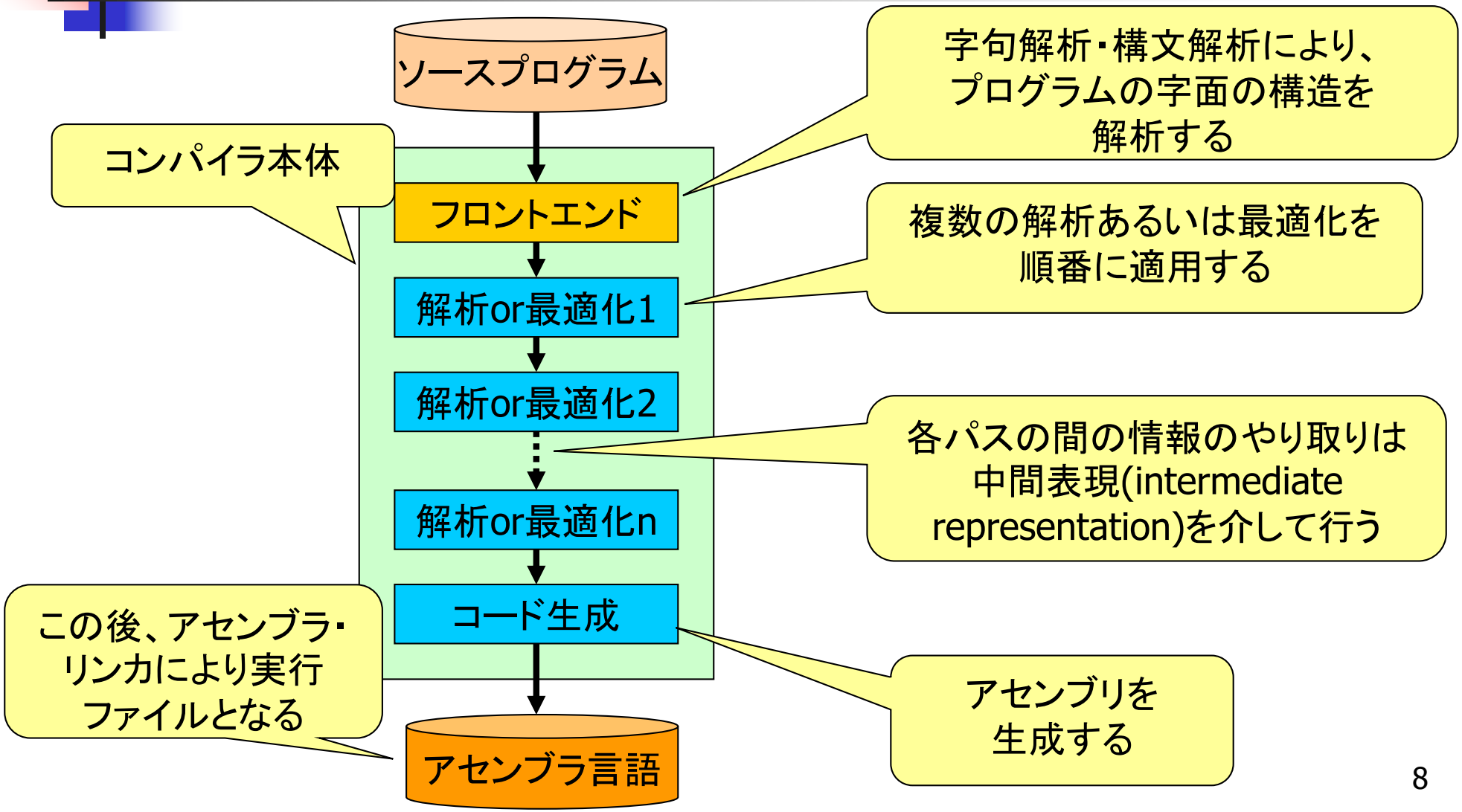
- 以下、進捗に応じて変更の場合有り
  - 4/24 字句解析／正則表現・Lex (Flex)
  - 5/1 構文解析／文脈自由文法・下向き構文解析
  - 5/8 構文解析／LR解析法 (1)
  - （5/15 Prof. Jeremy Gibbons講義）
  - 5/15 構文解析／LR解析法 (2)・Yacc (Bison)/内部表現基礎
  - 5/29 コード生成・簡易コンパイラ
  - 6/5　中間試験（字句解析・構文解析）
  - 6/12 関数呼び出し規約
  - 6/19 実行時メモリ管理・簡単なレジスタ割り当て
  - 7/17 理解度確認試験（最適化・コード生成）

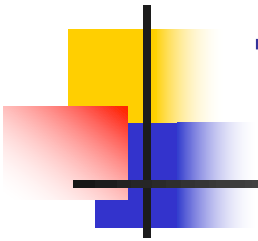


字句解析・構文解析

- コンパイラのフロントエンド部の処理
- ソースプログラムはファイル上の単なる文字列
  - 文字列から「書いてあること」を解析する必要がある
- 字句解析（今回の分）
  - ソースプログラムから単語を切り出す
- 構文解析（次回と次々回の分）
  - 式の解析が大きな動機の一つ
  - 字句解析で切り出した単語のつながりから、構文規則(syntax)をチェックする
    - 字句間のつながり方（式の場合、演算順序も表現する）をチェックする
  - さらにプログラムを内部表現に落とし込む

コンパイラの基本構成


ソースプログラム
コンパイラ本体
フロントエンド
解析or最適化1
解析or最適化2
解析or最適化n
コード生成
アセンブラ言語
字句解析・構文解析により、
プログラムの字面の構造を
解析する
複数の解析あるいは最適化を
順番に適用する
各パスの間の情報のやり取りは
中間表現(intermediate
representation)を介して行う
アセンブリを
生成する
この後、アセンブラ・
リンカにより実行
ファイルとなる



フロントエンド

- ソースプログラムを読み取る部分
- 字句解析 (lexical analyser, lexer, scanner)
  - ソースプログラムから単語(token)を切り出す
- 構文解析 (parser)
  - 字句解析で切り出した単語のつながりから、構文規則(syntax)をチェックする
  - ついでにプログラムを内部表現に落とし込む
- 文法規則から字句解析器・構文解析器を自動生成するツールがあります
  - lex (flex), yacc (bison), ANTLR

フロントエンドの動作イメージ

サンプルコード

```
int
foo ( int a )
{
    int val ;

    if ( a > 0 ) {
        val = 10 * ( a + 3 ) ;
    } else {
        val = 10 * a + 3 ;
    }

    return  val ;
}
```

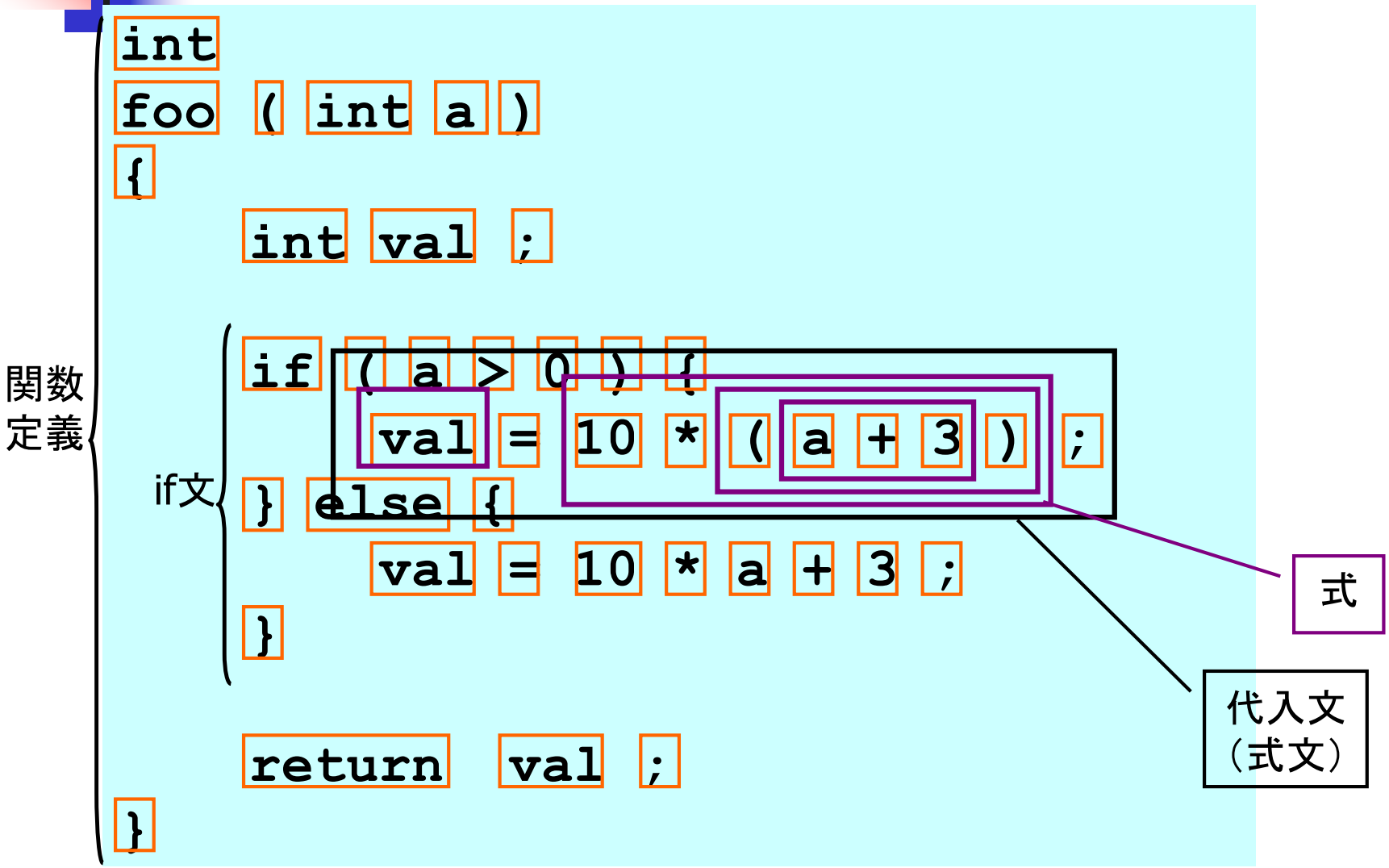
字句解析のイメージ
プログラムから単語を切り出す

```
int
foo ( int a )
{
    int val ;

    if ( a > 0 ) {
        val = 10 * ( a + 3 ) ;
    } else {
        val = 10 * a + 3 ;
    }

    return  val ;
}
```

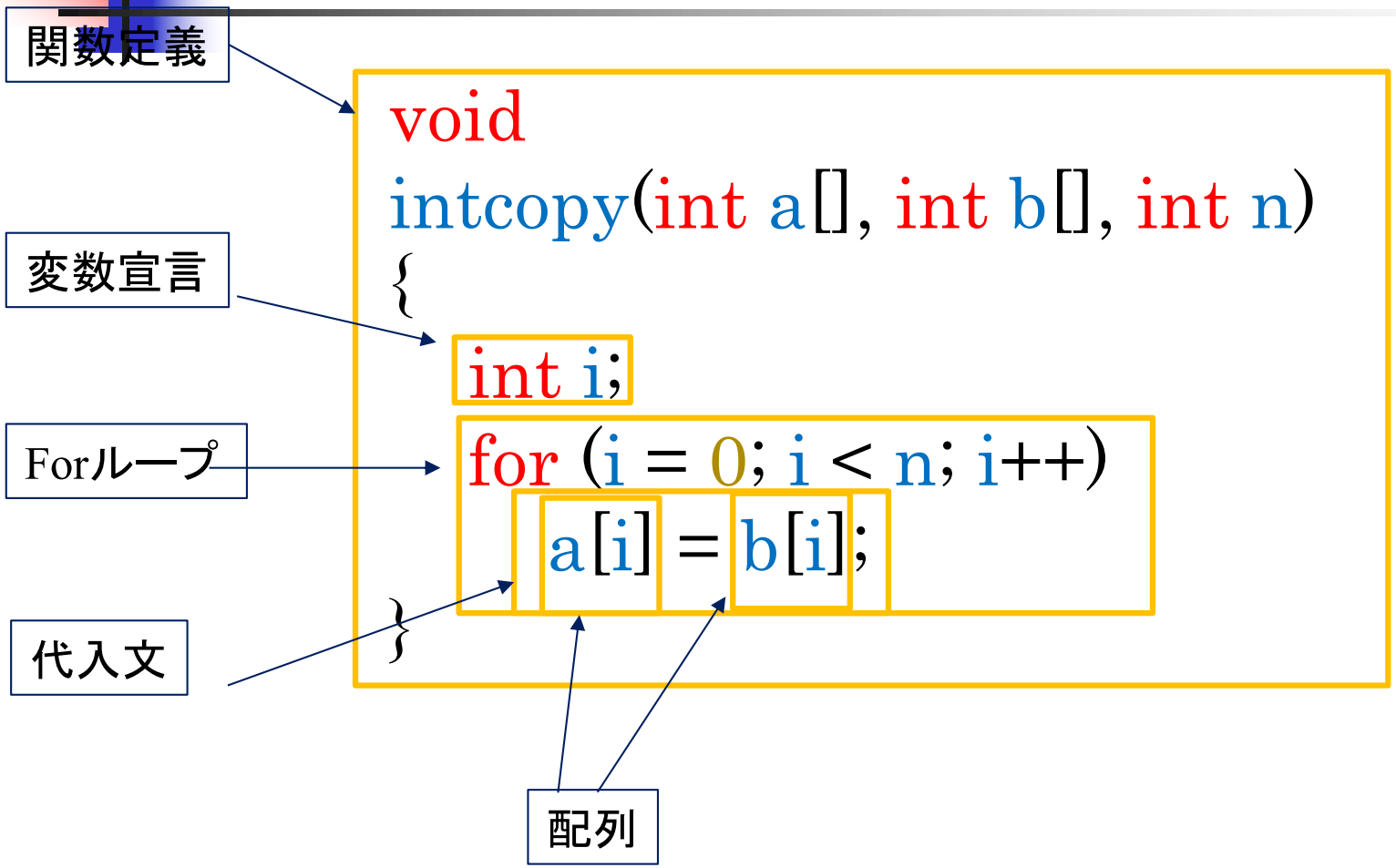
構文解析のイメージ
単語間の関係を解析する



字句解析：ソースプログラムを字句（トークン）に分解

```
void
intcopy(int a[], int b[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        a[i] = b[i];
}
```

ソースプログラムの構文解析



構文解析後の処理
中間表現への変換

右はgccによるforループ部分の
実際の中間表現 (gimple)

ソースプログラムとの共通部分は？
違う部分は？

```
<bb 2> :
i_10 = 0;
goto <bb 4>; [INV]

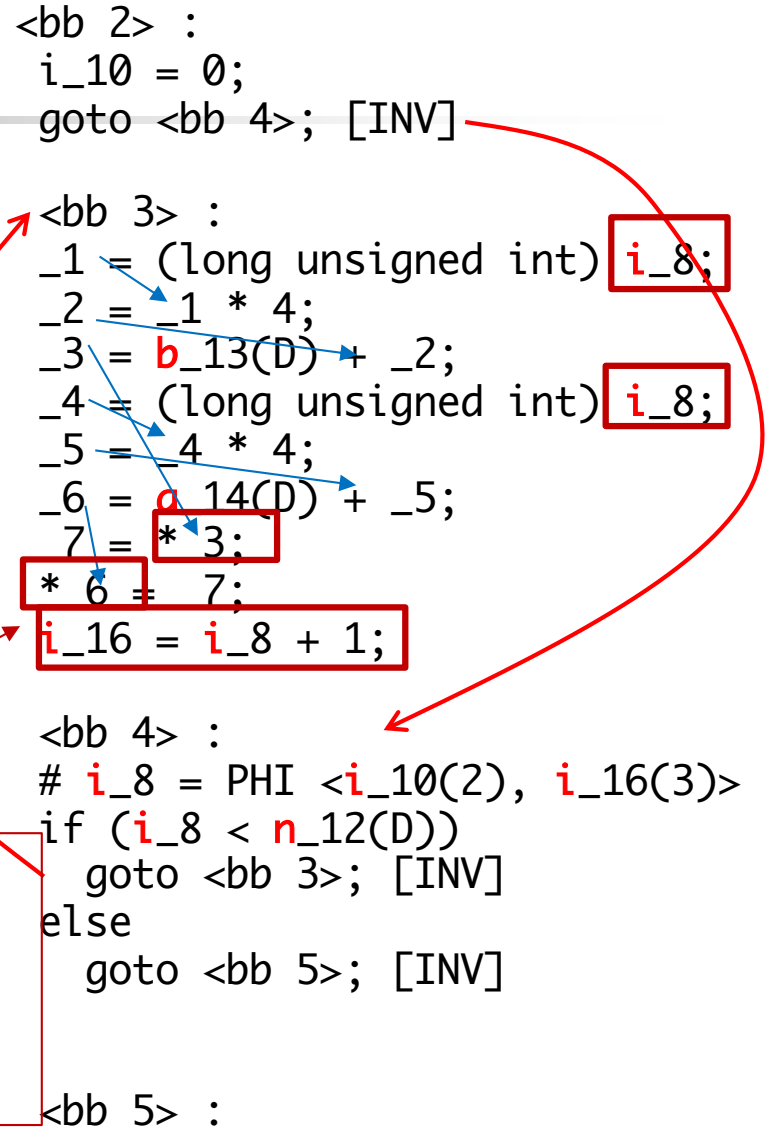
<bb 3> :
_1 = (long unsigned int) i_8;
_2 = _1 * 4;
_3 = b_13(D) + _2;
_4 = (long unsigned int) i_8;
_5 = _4 * 4;
_6 = a_14(D) + _5;
_7 = *_3;
*_6 = _7;
i_16 = i_8 + 1;

<bb 4> :
# i_8 = PHI <i_10(2), i_16(3)>
if (i_8 < n_12(D))
  goto <bb 3>; [INV]
else
  goto <bb 5>; [INV]


<bb 5> :
```

コード最適化のための プログラム解析

ここはループ



- “i” はループ1回転毎に1増える
- “ _3”と“_6”は “i*4”の要素を持つ
- つまり、“_3”と“_6”はループ1回転毎に4ずつ増える

なぜこの性質が大事か？

コード最適化とコード生成

```
<bb 2> :
 i_10 = 0;
 goto <bb 4>; [INV]

 <bb 3> :
 _1 = (long unsigned int) i_8;
 _2 = _1 * 4;
 _3 = b_13(D) + _2;
 _4 = (long unsigned int) i_8;
 _5 = _4 * 4;
 _6 = a_14(D) + _5;
 _7 = *_3;
 *_6 = _7;
 i_16 = i_8 + 1;

 <bb 4> :
 # i_8 = PHI <i_10(2), i_16(3)>
 if (i_8 < n_12(D))
   goto <bb 3>; [INV]
 else
   goto <bb 5>; [INV]


 <bb 5> :
```



```
<bb 3> [local count: 955630225]:
# ivtmp.4_7 = PHI <0(2), ivtmp.4_6(3)>
_5 = MEM[base: b_10(D), index: ivtmp.4_7,
         step: 4, offset: 0B];
MEM[base: a_11(D), index: ivtmp.4_7,
         step: 4, offset: 0B] = _5;
ivtmp.4_6 = ivtmp.4_7 + 1;
i_15 = (int) ivtmp.4_6;
if (n_9(D) > i_15)
  goto <bb 3>; [89.00%]
else
  goto <bb 4>; [11.00%]

<bb 4> [local count: 118111600]:
```



```
.L3:
        ldr     w4, [x1, x3, lsl 2]
        str     w4, [x0, x3, lsl 2]
        add     x3, x3, 1
        cmp     w2, w3
        bgt     .L3
.L1:
```

最適化無しのときの生成コード

```
.L3:
        ldrsw   x0, [sp, 44]
        lsl     x0, x0, 2
        ldr     x1, [sp, 16]
        add     x1, x1, x0
        ldrsw   x0, [sp, 44]
        lsl     x0, x0, 2
        ldr     x2, [sp, 24]
        add     x0, x2, x0
        ldr     w1, [x1]
        str     w1, [x0]
        ldr     w0, [sp, 44]
        add     w0, w0, 1
        str     w0, [sp, 44]
.L2:
        ldr     w1, [sp, 44]
        ldr     w0, [sp, 12]
        cmp     w1, w0
        blt     .L3
```

木村担当部分の実習では，
最適化無しのときのコンパイラ
の処理を扱います．

実習用機材（Raspberry Pi4）の配布

- 本日この後、55N号館5階507A室にて配布しますので、そちらに向かってください
- 一人一台です。他の人の代理で受け取ることは禁止します
- 混雑を避けるため、4グループに分けて10分毎に部屋に向かってもらいます。
  - グループ1: 過年度、学系II ～1W242100まで
  - グループ2: 1W242101～1W242305まで
  - グループ3: 1W242306～、学系III ～1W243025まで
  - グループ4: 1W243026～